



Published using Google Docs

[Report abuse](#)[Learn more](#)

Building a "Living Terminal" GitHub Profile README

Updated automatically every 5 minutes

Building a "Living Terminal" GitHub Profile README

A walkthrough for turning your GitHub profile into something that looks alive: a portrait that draws itself in ASCII, a system-info style panel next to it, and a contribution graph that animates in and updates on its own every day — all without a single external badge service, API token, or line of client-side JavaScript.

The core trick

GitHub's markdown renderer is strict. It strips `<script>` tags, ignores most inline `style` attributes, and won't let you run arbitrary CSS on the page. But there's one loophole: if you embed an SVG file via a normal `<img>` tag, GitHub renders it as an image — and that image is allowed to contain its own animation logic (SMIL tags or embedded `<style>` keyframes), because from GitHub's point of view it's just a static image file that happens to move.



Published using Google Docs

[Report abuse](#)[Learn more](#)

Building a "Living Terminal" GitHub Profile README

Updated automatically every 5 minutes

What you'll end up with

File	What it does
<code>portrait.svg</code>	Your photo, converted to monochrome ASCII characters that draw in top-to-bottom
<code>sysinfo.svg</code>	A terminal "system info" style panel that lists your focus, tools, and a highlight or two
<code>graph.svg</code>	Your actual contribution calendar, redrawn as a grid that animates into place
<code>README.md</code>	Lays the three out like a terminal session
A GitHub Actions job	Re-pulls your contribution data and re-renders the graph once a day

Step 1 — The one repo that matters

GitHub treats a repository with the exact same name as your account



Published using Google Docs

[Report abuse](#)[Learn more](#)

Building a "Living Terminal" GitHub Profile README

Updated automatically every 5 minutes

```
mkdir -p tools assets  
.github/workflows
```

Everything you build below lives inside `tools/` and gets committed as flat SVG/PNG assets in the repo root or `assets/`.

Step 2 — Two separate dependency sets

Split your requirements so the daily automation stays lightweight — you don't want image-processing libraries installed just to refresh a graph:

```
# tools/requirements-daily.txt (used  
by the cron job)  
httpx==0.27.0  
lxml==5.2.2
```

```
# tools/requirements-art.txt (only  
used when you redo your portrait)  
Pillow  
numpy  
opencv-python  
rembg
```

Install whichever set you need locally with a venv:

```
python -m venv .venv && source  
.venv/bin/activate  
pip install -r tools/requirements-  
art.txt
```



Published using Google Docs

[Report abuse](#)[Learn more](#)

Building a "Living Terminal" GitHub Profile README

Updated automatically every 5 minutes

Split this into two stages so you're not redoing image cleanup every time you tweak the animation.

3a. Clean up the source photo

Straight-out-of-camera photos usually convert to mush — everything ends up mid-gray. Three fixes, in order:

1. **Cut the background** with `rembg` so only the subject remains.
2. **Even out lighting** with OpenCV's CLAHE (adaptive histogram equalization), which pulls real shadow/highlight detail out of a flat-lit face.
3. **Drop it onto a white canvas** so the background falls at the light end of your character ramp instead of the dark end.

```
python tools/clean_photo.py my-photo.jpg
# writes assets/photo-ready.png
```

3b. Map pixels to characters and animate

Downscale the cleaned image to a small character grid and pick a glyph per cell based on brightness. Here's a ramp that reads a bit softer than the usual `@%#` block style:



Published using Google Docs

[Report abuse](#)[Learn more](#)

Building a "Living Terminal" GitHub Profile README

Updated automatically every 5 minutes

- **Single fill color.** Multi-color ASCII art almost always reads as visual clutter rather than a portrait — pick one accent color and stick to it.
- **Aggressive contrast during cleanup.** If the background isn't cleanly white, it'll print as faint dots instead of empty space.

For the motion, wrap each output row in its own clipping rectangle and animate the clip's width from 0 to full, staggering the start time row by row (roughly 40ms apart works well). Once it reaches the last row, let it hold — a portrait that loops forever gets distracting fast.

```
python tools/render_portrait.py
# writes portrait.svg
```

Step 4 — A terminal-style info panel

Rather than trying to make this look like a real CLI tool's actual output, treat it as its own small design: a header bar, then a handful of labeled rows (something like `role`, `focus`, `stack`, `now`). This is the place for context your contribution graph can't show — what you're actually working on, not just how often you commit.

```
# tools/render_panel.py
ROWS = [
    ("role", "Software Engineer"),
    ("focus", "Distributed Systems"),
    ("stack", "Go · Rust · Postgres"),
    ("now", "Building a job-queue from scratch"),
]
```



Published using Google Docs

[Report abuse](#)[Learn more](#)

Building a "Living Terminal" GitHub Profile README

Updated automatically every 5 minutes

```
PREVIEW=1 python  
tools/render_panel.py  
python tools/render_panel.py #  
writes sysinfo.svg
```

Step 5 — The contribution graph, kept alive

This is the piece that needs to update daily, so it's built in two stages: pull the data, then draw it.

5a. Pull your contributions without a token

You don't need OAuth or a personal access token for this — GitHub serves a plain HTML fragment of your contribution calendar at:

<https://github.com/users/<yourusername>/contributions>

It's the same markup the profile page itself consumes. Fetch it, parse the day cells out with an HTML parser, and save the results:

```
python tools/pull_contributions.py  
# writes assets/contributions.json
```

Store more than raw counts while you're at it — current streak, longest streak, and a busiest-day-of-week breakdown make the eventual footer more interesting than just a total.



Published using Google Docs

[Report abuse](#)[Learn more](#)

Building a "Living Terminal" GitHub Profile README

Updated automatically every 5 minutes

color ramp — no need to match GitHub's exact greens if you want the graph to match your portrait's accent color instead:

```
LEVELS = ["#1a1a2e", "#16537e",
"#1c7ed6", "#4dabf7", "#a5d8ff"]
# index 0 = no activity, index 4 =
your top activity tier
```

For the reveal, animate the squares in by column (week) rather than the more common per-row wipe — it reads a little more like a wave. Add a small legend and a one-line stats summary underneath, then let it freeze once fully drawn.

```
python tools/render_graph.py
# writes graph.svg
```

Step 6 — Assembling the README

The README's only job now is layout. A `<table>` is still the most reliable way to get two SVGs sitting side-by-side on GitHub's renderer:

```
<div align="center">

<h3><code>$ cat
contributions.log</code></h3>


<br><br>

<h3><code>$ whoami --
verbose</code></h3>
<table>
  <tr>
    <td valign="top">
</td>
    <td valign="top">
</td>
  </tr>
</table>
```



Published using Google Docs

[Report abuse](#)[Learn more](#)

Building a "Living Terminal" GitHub Profile README

Updated automatically every 5 minutes

- **Inline spacing is ignored.** `style="margin-top:..."` does nothing on GitHub's markdown — use `<br>` tags for vertical gaps instead.
- **`<h1>/<h2>` add a full-width line under them.** Use `<h3>` (or lower) if you just want a label without a divider.
- **All motion has to live inside the SVGs themselves** — there's no external CSS or JS to lean on.

Step 7 — Automating the daily refresh

Only the contribution graph actually needs to change day-to-day, so the workflow only touches that piece:

```
# .github/workflows/refresh-
graph.yml
name: Refresh contribution graph

on:
  schedule:
    - cron: "0 5 * * *" # once a day,
      early morning UTC
    workflow_dispatch: {}

permissions:
  contents: write

jobs:
  refresh:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
```



Published using Google Docs

[Report abuse](#)[Learn more](#)

Building a "Living Terminal" GitHub Profile README

Updated automatically every 5 minutes

```
commit-action@v5
with:
  commit_message: "auto:
refresh contribution graph [skip ci]"
  file_pattern:
"assets/contributions.json graph.svg"
```

`[skip ci]` stops the bot's own commit from re-kicking the workflow. Run it once manually from the Actions tab (`workflow_dispatch`) to confirm it commits a fresh SVG before you leave it on the cron.

Why generate your own SVGs instead of using a stats badge

Hosted profile-stats widgets are convenient, but they're someone else's server: they can rate-limit you, go down and leave a broken-image icon on your profile, or change their styling without warning. Rendering your own SVGs means the art lives in your repo, loads instantly, looks exactly the way you designed it, and the only outside dependency is a public GitHub HTML endpoint that needs no authentication at all.

Recap

1. Create the `<username>/<username>` repo — its README is your profile page.
2. Clean up a photo, then convert it into a self-drawing monochrome ASCII SVG.



Published using Google Docs

[Report abuse](#)[Learn more](#)

Building a "Living Terminal" GitHub Profile README

Updated automatically every 5 minutes

animated grid.

5. Lay all three out in a centered, terminal-themed README.
6. Automate the graph refresh with a daily GitHub Actions cron.